

Automatismes Série 2

A3 Puissances

A4 Écritures d'un nombre

Voir Séquence 1 et rappels Automatismes Série 1.

A9 Développer, factoriser

À retenir

1. Distributivité simple

Pour tout nombre k et tous nombres a, b :

$$k(a + b) = k \times a + k \times b$$

2. Double distributivité

Pour tous nombres a, b, c et d :

$$(a + b)(c + d) = ac + ad + bc + bd$$



Exemple

$$\begin{aligned} A(x) &= 3(x + 5) \\ &= 3 \times x + 3 \times 5 \\ &= 3x + 15 \end{aligned}$$

$\times$	x	$+5$
3	$3x$	15

$$\begin{aligned} B(x) &= (2x - 1)(x + 4) \\ &= 2x^2 + 8x - x - 4 \\ &= 2x^2 + 7x - 4 \end{aligned}$$

$\times$	x	$+4$
$2x$	$2x^2$	$8x$
-1	$-x$	-4

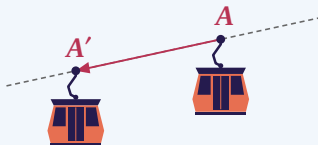
Translations et vecteurs

À retenir

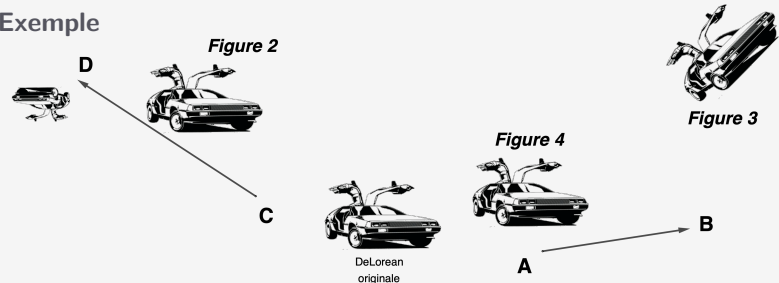
Une **translation** est une transformation qui permet de faire **glisser** une figure parallèlement à une droite, dans un sens donné.

L'image n'est ni retournée, ni déformée.

On peut schématiser ce glissement par une flèche appelée **vecteur**.



Exemple



La figure 4 est l'image de la figure originale par la translation qui transforme A en B.
La figure 2 est l'image de la figure originale par la translation de vecteur $\overrightarrow{DC}$.
Les figures 1 et 3 ne sont pas des images par translation : une translation conserve la taille et l'orientation.

Algo

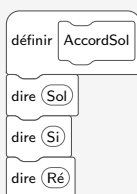
À retenir

Un **bloc personnalisé** (Scratch) ou une **fonction** (Python) permet de regrouper plusieurs instructions sous un même nom, pour les réutiliser en un seul appel.

En Python, on la définit avec `def nom() :` suivi d'une ligne indentée, puis on l'exécute en écrivant dans la console `nom()`.



Exemple



Sol, Si, Ré

```
def AccordSol():
    return "Sol", "Si", "Ré"
```

```
>>> AccordSol()
('Sol', 'Si', 'Ré')
```

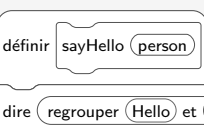
À retenir

Parfois, un bloc a besoin d'une information précise pour exécuter son code : c'est un **paramètre**.

En Python, on l'écrit entre parenthèses : `def nom(param) :` ; l'appel `nom(valeur_voulue)` dans la console remplace `param` par `valeur` dans le code exécuté.



Exemple



Hello John!

```
def sayHello(person):
    return "Hello " + person + "!"
```

```
>>> sayHello("John")
'Hello John!'
```

		A3 Puissances	A4 Écritures d'un nombre	Vecteurs Égalité, somme, translations	A9 Développer Factoriser	Algo
Jour 1 Date : _____	QCM			①	②	
	Questions	③		④	⑤	
Jour 2 Date : _____	QCM			①		②
	Questions	③	④	⑤		
Jour 3 Date : _____	QCM			①	②	
	Questions		③			④
Jour 4 Date : _____	QCM	②		①		
	Questions		③		④	⑤